

PROJETO 03 DE 10

Ferramenta DevOps em Go — CLI de Monitoramento

Intermediário

Go · Goroutines · CLI · Docker

Guia de estudo DevOps & Cloud · Junho de 2026

Projeto 3 — Ferramenta DevOps em Go — CLI de Monitoramento

Intermediário

Uma ferramenta de linha de comando, escrita em Go, que verifica a saúde de vários serviços ao mesmo tempo (em paralelo) e reporta o status. Seu diferencial.

Go

Goroutines

CLI

Docker

O que o projeto faz

Você roda `./healthcheck -urls=site1.com,site2.com` e a ferramenta checa cada URL **ao mesmo tempo** (usando goroutines, a concorrência do Go), medindo o tempo de resposta e dizendo quais estão no ar. É o tipo de ferramenta que times de DevOps realmente constroem.



O que é usado (stack)

- **Go** — linguagem compilada, rápida, usada em Docker/Kubernetes.
- **Goroutines + channels** — concorrência (o grande diferencial do Go).
- **net/http** — fazer as requisições.
- **flag** — ler argumentos da linha de comando.
- **Docker** — empacotar a ferramenta (opcional).

Estrutura do projeto

```
go-healthcheck/  
├─ go.mod      # define o módulo e a versão do Go  
├─ main.go     # todo o programa  
└─ Dockerfile  # empacota o binário (opcional)
```

Código comentado (main.go)

```
package main

import (
    "flag"      // ler argumentos da linha de comando
    "fmt"       // imprimir na tela
    "net/http"  // fazer requisições HTTP
    "strings"   // dividir a string de URLs
    "sync"      // esperar todas as goroutines (WaitGroup)
    "time"      // medir o tempo de resposta
)

// checa UMA url e manda o resultado para a tela.
func checar(url string, wg *sync.WaitGroup) {
    defer wg.Done() // avisa o WaitGroup ao terminar
    inicio := time.Now()
    resp, err := http.Get(url) // faz a requisição
    dur := time.Since(inicio) // quanto tempo levou
    if err != nil {
        fmt.Printf("[FORA] %s (%v)\n", url, err)
        return
    }
    defer resp.Body.Close() // fecha a conexão (evita vazamento)
    fmt.Printf("[OK] %s %d %v\n", url, resp.StatusCode, dur)
}

func main() {
    lista := flag.String("urls", "", "URLs separadas por vírgula")
    flag.Parse() // lê o que veio na linha de comando

    urls := strings.Split(*lista, ",")
    var wg sync.WaitGroup // contador de goroutines
    for _, u := range urls {
        wg.Add(1) // +1 antes de cada goroutine
        go checar(u, &wg) // 'go' = roda em paralelo
    }
    wg.Wait() // espera todas terminarem
}
```

As linhas que ensinam Go: `go checar(...)` dispara a função numa **goroutine** — todas as URLs são checadas simultaneamente, não uma de cada vez. O `sync.WaitGroup` é o que faz o `main` esperar todas terminarem: `wg.Add(1)` soma, `wg.Done()` subtrai, e `wg.Wait()` bloqueia até zerar. O `defer` agenda algo para o fim da função (fechar a resposta) — padrão clássico de Go. O `if err != nil` é o tratamento de erro idiomático da linguagem.

Comandos

```
go mod init go-healthcheck # cria o go.mod
go run main.go -urls=https://google.com,https://github.com
go build -o healthcheck # gera um binário standalone
./healthcheck -urls=...
```

Explicação linha a linha — `main.go`

1	<code>package main</code>	Todo programa executável em Go vive no pacote main .
2	<code>import (...)</code>	Importa os pacotes usados (flag, fmt, net/http, strings, sync, time).
3	<code>func checar(url string, wg *sync.WaitGroup) {</code>	Função que checa UMA URL. Recebe um ponteiro para o WaitGroup para avisá-lo ao terminar.
4	<code>defer wg.Done()</code>	defer agenda esta chamada para o fim da função: avisa o WaitGroup que terminou.
5	<code>inicio := time.Now()</code>	Marca o instante antes da requisição (para medir a duração).
6	<code>resp, err := http.Get(url)</code>	Faz a requisição HTTP. Go retorna dois valores: a resposta e um erro.
7	<code>dur := time.Since(inicio)</code>	Calcula quanto tempo passou desde o início.
8	<code>if err != nil {</code>	Tratamento de erro idiomático do Go: se deu erro...
9	<code>fmt.Printf("[FORA] %s\n", url); return</code>	...imprime que o serviço está fora e sai da função.
10	<code>}</code>	Fim do if de erro.

11	<code>defer resp.Body.Close()</code>	Agenda o fechamento da conexão — evita vazamento de recursos.
12	<code>fmt.Printf("[OK] %s %d %v\n", ...)</code>	Imprime status OK, código HTTP e o tempo de resposta.
13	<code>}</code>	Fim da função <code>checar</code> .
14	<code>func main() {</code>	Ponto de entrada do programa.
15	<code>lista := flag.String("urls", "", "...")</code>	Declara um argumento de linha de comando chamado <code>-urls</code> .
16	<code>flag.Parse()</code>	Lê de fato os argumentos passados no terminal.
17	<code>urls := strings.Split(*lista, ",")</code>	Quebra a string de URLs em uma lista, separando por vírgula.
18	<code>var wg sync.WaitGroup</code>	Cria o contador que vai esperar todas as goroutines.
19	<code>for _, u := range urls {</code>	Percorre cada URL da lista.
20	<code>wg.Add(1)</code>	Soma 1 ao contador ANTES de iniciar a goroutine.
21	<code>go checar(u, &wg)</code>	go dispara <code>checar()</code> em paralelo (uma goroutine por URL).
22	<code>}</code>	Fim do loop.
23	<code>wg.Wait()</code>	Bloqueia o programa até todas as goroutines terminarem.
24	<code>}</code>	Fim do <code>main</code> .

Erros comuns na criação

- **O programa termina sem mostrar nada:** você esqueceu o `wg.Wait()` — o `main` encerra antes das goroutines.
- **panic: nil pointer:** usou `resp.StatusCode` sem checar o `err` antes. Sempre cheque o erro primeiro.
- **too many open files:** esqueceu `resp.Body.Close()` — conexões vazam.
- **imported and not used:** Go é rígido — todo `import` precisa ser usado, senão não compila.
- **URL sem http://:** o `http.Get` exige o esquema. Valide/normalize a entrada.

Por que impressiona o recrutador

Une **Go** (sua meta) com **DevOps**, e é original — não é o típico CRUD de tutorial. Mostra que você *constrói ferramenta*, entende concorrência e linha de comando.

Para aprender mais

- Estude **channels** para coletar resultados de forma mais elegante que imprimir direto.
- Adicione um **timeout** com `http.Client{Timeout: ...}`.
- Recursos grátis: Tour of Go (go.dev/tour) e Go by Example.